Smart Attendance Platform

Full Technical Specification & Developer Knowledge Transfer Guide

Version 2.5.0 (August 2026)

This document provides a comprehensive technical guide to the architecture, database schema, REST APIs, WebSocket interfaces, liveness detection logic, and deployment steps of the Smart Attendance Platform.

Engineered as a three-tier system to prevent proxy attendance using local facial liveness comparison, dynamic rolling QR codes, and BLE proximity verification.

Author: BunnyBadri

System Host: FastAPI & SQLite Backend + Web Projector + Flutter Mobile App

Generated On: August 9, 2026

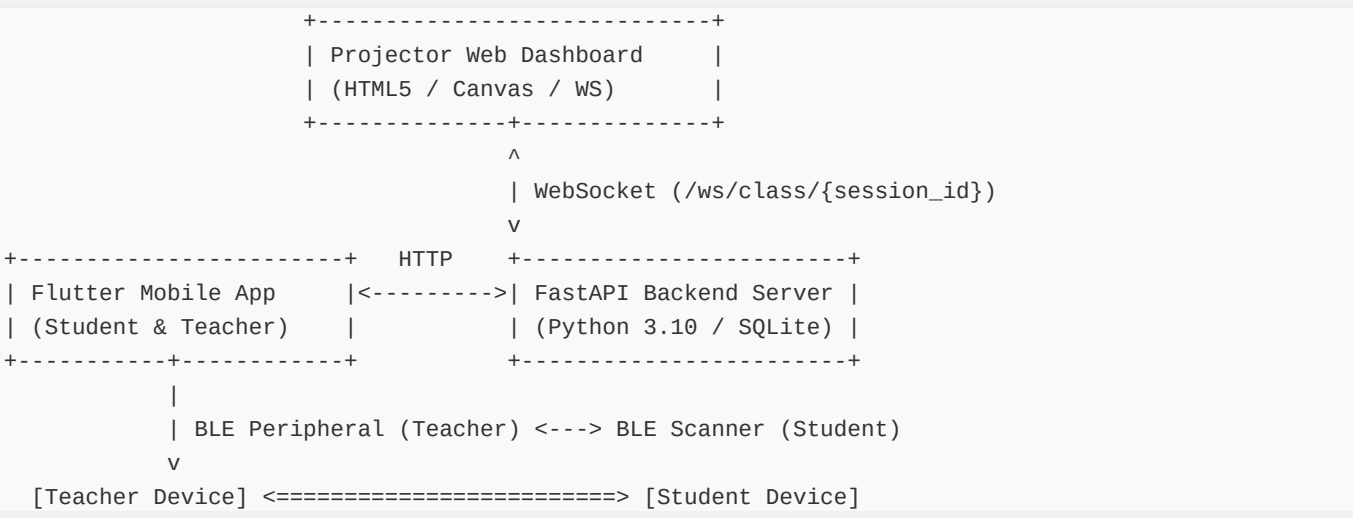
System Overview & Value Proposition

The **Smart Attendance Platform** is a 3-tier attendance system engineered to prevent proxy attendance (students signing in for absent peers) in academic environments.

Verification relies on three independent factors:

1. **Biometric Face Recognition & Interactive Liveness**: Google ML Kit local facial landmark vector comparison + blink & smile liveness validation.
2. **Dynamic Rolling QR Codes**: Server-generated TOTP tokens (refreshed every 10 seconds with a 1-step window) displayed on classroom projectors via WebSockets.
3. **Bluetooth Low Energy (BLE) Proximity**: Teacher device acts as a BLE peripheral broadcasting a session beacon; student device scans for signal strength ($>= -85 dBm$, ~8-10m).

System Architecture & Data Flow



Database Schema (`backend/models.py` & `backend/database.py`)

1. `users` Table

* "id" (INTEGER, Primary Key, Autoincrement)

* "username" (VARCHAR, Unique, Indexed)
* "name" (VARCHAR)
* "hashed_password" (VARCHAR - Bcrypt)
* "role" (VARCHAR - "'student'" or "'teacher'")
* "face_embedding" (TEXT - JSON String "{"r1": float, "r2": float}")

2. `classes` Table

* "id" (INTEGER, Primary Key)
* "name" (VARCHAR)
* "code" (VARCHAR, Unique - e.g., "'CS-101'")
* "teacher_id" (INTEGER, Foreign Key $->$ "users.id")

3. `enrollments` Join Table

* "student_id" (INTEGER, Foreign Key $->$ "users.id")
* "class_id" (INTEGER, Foreign Key $->$ "classes.id")

4. `sessions` Table

* "id" (INTEGER, Primary Key)
* "short_id" (VARCHAR - 6-character PIN for Projector pairing)
* "class_id" (INTEGER, Foreign Key $->$ "classes.id")
* "ble_uuid" (VARCHAR - UUID4 generated for teacher BLE advertising)
* "otp_secret" (VARCHAR - Base32 secret key for TOTP)
* "is_active" (BOOLEAN - Default "True")
* "created_at" (DATETIME)

5. `attendance` Table

* "id" (INTEGER, Primary Key)
* "session_id" (INTEGER, Foreign Key $->$ "sessions.id")
* "student_id" (INTEGER, Foreign Key $->$ "users.id")
* "verified_proximity" (BOOLEAN)
* "verified_face" (BOOLEAN)
* "status" (VARCHAR - "'present'")
* "timestamp" (DATETIME)

REST API & WebSocket Specifications (`backend/main.py`)

Authentication & User Management

* "POST /auth/register"
* Body: "{"username": "...", "password": "...", "name": "...", "role": "student"|"teacher", "face_embedding": "{\"r1\":1.23,\"r2\":1.25}"}"
* "POST /auth/login"
* Body: "{"username": "...", "password": "..."}"
* Returns JWT Bearer token + user payload.
* "GET /users/{user_id}/classes"
* Returns enrolled classes for student or taught classes for teacher.

Class & Session Management

* "POST /classes/create" (Teacher) $->$ Body: "{"name": "...", "code": "CS-101", "teacher_id": 1}"
* "POST /classes/enroll" (Student) $->$ Body: "{"student_id": 1, "class_code": "CS-101"}"
* "POST /sessions/start" (Teacher)
* Body: "{"class_id": 1}"
* Closes any active session for class, generates new "ble_uuid" and "otp_secret".
* "GET /sessions/active/{class_id}" $->$ Returns active session details if hosting.
* "POST /sessions/end" $->$ Closes session ("is_active = False").

Attendance & Real-Time Sync

* "POST /attendance/submit" (Student)
* Body: "{"session_id": 1, "student_id": 2, "otp_token": "123456", "verified_proximity": true, "verified_face": true}"
* Validates TOTP token ("pyotp.TOTP(secret, interval=10)" with 1-step window = 10s drift allowance).
* "GET /sessions/{session_id}/attendance" $->$ Returns real-time roster of present students.
* "WS /ws/class/{session_id}" (Projector Dashboard)
* Pushes "otp_update" event every second with "token" and "expires_in".
* Broadcasts "student_checkin" events in real-time when students check in.

Mobile App Architecture & Flow (`mobile/lib`)

1. `student_flow.dart` (Verification Wizard)

* **Step 0 (Face ID & Liveness)**:
* Camera stream parsed via ML Kit "FaceDetector".
* Prompts: **Blink** ("leftEyeOpenProbability < 0.2" & "rightEyeOpenProbability < 0.2") and **Smile** ("smilingProbability > 0.8").
* Landmark Ratios: $r_1 = d_{\text{eyes}} / d_{\text{leftEyeNose}}$, $r_2 = d_{\text{eyes}} /

d_{\text{rightEyeNose}}$.

* Similarity score threshold: $>= 80\%$.
* **Step 1 (Scan QR)**: Uses "mobile_scanner" to parse payload "{"session_id": 1, "token": "..."}".
* **Step 2 (BLE Proximity)**: Uses "flutter_blue_plus". Scans for 8 seconds. Checks "advName", "localName", "platformName", and "serviceUuids" for "SmartAtt". Signal threshold: RSSI $>= -85 dBm$.
* **Step 3 (Submission)**: Submits POST request to "/attendance/submit".

2. `teacher_home.dart` (Session Host)

* Starts session via API.
* Starts BLE Peripheral advertising via "flutter_ble_peripheral": "serviceUuid" = session "ble_uuid", "localName" = "SmartAtt_[CLASS_CODE]", "includeDeviceName: true".
* Displays Session PIN for projector dashboard pairing and polls check-ins every 3 seconds.

Build Configuration & Critical Gotchas

1. Android Release Build & R8 Minification (ProGuard)

* R8 minification is active in release builds ("isMinifyEnabled = true" in "mobile/android/app/build.gradle.kts").
* Keep rules in "mobile/android/app/proguard-rules.pro" preserve ML Kit and BLE native plugin reflection:

```
-keep class com.google.mlkit.** { *; }
-keep interface com.google.mlkit.** { *; }
-dontwarn com.google.mlkit.**
-keep class com.google.android.gms.internal.mlkit_** { *; }
-dontwarn com.google.android.gms.internal.mlkit_**
-keep class com.google.android.gms.** { *; }
-keep class com.bosoco.flutter_blue_plus.** { *; }
-dontwarn com.bosoco.flutter_blue_plus.**
-keep class com.github.jasonross.flutter_ble_peripheral.** { *; }
-dontwarn com.github.jasonross.flutter_ble_peripheral.**
-keep class dev.flutter.plugins.** { *; }
-keepclasseswithmembernames class * { native <methods>; }
```

2. Android 12+ BLE Scanning Permission Flag

* Do **NOT** add "android:usesPermissionFlags="neverForLocation"" to "BLUETOOTH_SCAN" in "AndroidManifest.xml".
* Setting "neverForLocation" causes Android OS to drop BLE beacon scan results.

How to Run & Develop Locally

1. Backend Server

```
cd backend
pip install -r requirements.txt
uvicorn main:app --reload --host 0.0.0.0 --port 8000
```

2. Mobile App

```
cd mobile
flutter pub get
flutter build apk --release
```

3. Projector Web Dashboard

```
cd web
npx serve .
```